Coditu — emergency card

Print this — one sheet, both sides. For the night something is gone and nobody remembers any of it.

Written 2026-08-31, and **verified by following it literally with the network off**. The first draft was wrong in three places; each is fixed below and recorded in `docs/06-PROGRESS.md`.

1. Your data is not inside the application

It is a nightly encrypted archive, ~19 MB, and there are several copies. *The app is down* and *the data is gone* are different problems and only the first has ever been likely.

Getting the rows back takes about a minute and needs nobody's account. Getting the site back is a separate, slower job — §5. Do the first one first.

2. Where the copies are

Copy	Where	Needs
PRIMARY	Mohamed's Mac, `~/Library/Application Support/coditu-backups/`	The Mac
Off-site 1	The Google Drive folder named as `OFFSITE_drive_DIR` in `~/.config/coditu/nightly.env`	A Google sign-in, or any machine syncing that Drive
Off-site 2	An external disk holding `.coditu-backup-volume`	The disk
Older sets	`MooTamer/coditu-recovery` releases — **up to 2026-08-31 only**	A GitHub sign-in

Each night is two files: `<id>.tar.gz.age` (encrypted) and `<id>.metadata.json` (its checksum and counts). `<id>` is `YYYY-MM-DDTHHMMSSZ`. **Sort by name, never by date — newest name wins.**

3. The key — read this before anything else fails

Encrypted with **age**. Without the private key the archives are noise. There is no escrow, no recovery, no reset.

Where it is	`~/.config/coditu/backup.agekey` on Mohamed's Mac, mode 0600
Second copy	□ *fill this in* — the safe, the wallet, the password-manager entry
Public recipient (not secret)	`ops/backup-recipient.txt` in the repository

> **If there is no second copy and the Mac is gone, every archive everywhere is unreadable.** Make it today, and **not where the archives are** — a key beside its ciphertext is not a key. Paper in another building. A password manager. Anywhere the Drive folder is not.

Never type the key into a chat, a ticket, a commit, or this card.

4. Get the data back

Any machine with Docker and `age`. **No network is needed if the image is already pulled** — it is, on Mohamed's Mac.

```
brew install age                                  # or: apt install age
ID=<newest id>                                    # e.g. 2026-08-31T070134Z
mkdir -p /tmp/coditu-restore
age -d -i /path/to/backup.agekey "$ID.tar.gz.age" | tar -xz -C /tmp/coditu-restore
```

Inside: `schema.sql`, `data.sql`, `roles.sql`, `row-counts.tsv`, `manifest.json`, and `storage/`.

The target must be Supabase's Postgres, not a plain one. Measured: plain `postgres:17` throws 545 errors — the dump refers to `auth`, `storage` and `extensions` schemas it does not have.

```
docker run -d --name coditu-restore -e POSTGRES_PASSWORD=postgres -p 5599:5432 \
  public.ecr.aws/supabase/postgres:17.6.1.155
sleep 15
U='postgresql://postgres:postgres@127.0.0.1:5599/postgres'
psql -X -q "$U" -f /tmp/coditu-restore/roles.sql        # a few "role does not exist" — fine
psql -X -q "$U" -f /tmp/coditu-restore/schema.sql       # clean
psql -X -q "$U" -f /tmp/coditu-restore/data.sql         # see the note below
psql -X -qAt "$U" -c 'select count(*) from public.work_items'
```

What this gives you, measured rather than hoped: every application table, row for row — `public` **35 of 35 tables, 1,184 of 1,184 rows**. What it does not give you is `auth` and `storage` *metadata*: the public image trails hosted, so `data.sql` errors on tables it lacks (`storage.vector_indexes` and similar) and the `auth.users` rows after it are skipped. Those need a target as new as hosted — the full procedure is `verify-backup-restore.sh` in `MooTamer/coditu-recovery`, and `docs/12-RECOVERY-RUNBOOK.md` §3.

The Storage files — avatars and attachments — are already extracted under `/tmp/coditu-restore/storage/`**.** They are ordinary files. They need no database and no tool.

5. Get the application back

Only if you need the app, not just the data. Estimate **30–60 minutes**, and `docs/14-DEPENDENCY-LEDGER.md` says what that is made of.

1. Create a new Supabase project; note its URL, anon key and service-role key.
2. Restore as in §4, against the new project's connection string.
3. **Reapply the two triggers no dump carries** — without them there is no profile creation and **no sign-up gate**: `scripts/reapply-managed-triggers.sh '<new connection string>'`
4. Set `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY` on Vercel and redeploy — or `npm run build && npm start` anywhere.

6. Check you really got it

```
node scripts/check-materialised.mjs /path/to/the/archives     # materialised=true
diff <(sort /tmp/coditu-restore/row-counts.tsv) ...           # per-table truth, in the archive
```

`materialised=true` means every archive **read back byte-for-byte** and matched its own sidecar. **A file of the right size is not evidence:** a cloud folder can hold a placeholder with the right name, the right size and no bytes at all, which reads only when the network is up.

7. How long it takes — measured 2026-08-31 with the wifi off

No default route; `api.github.com`, `registry.npmjs.org` and `google.com` all failed to resolve.

Verify all 8 archives (155 MB) read back and hash	**under 1 s**
§4 end to end — decrypt, extract, start Postgres, load all three files	**4 s**
Result	`public` **35/35 tables, 1,184/1,184 rows**, and **135** Storage files
Read the Storage files	immediate — they are files
Stand the application up	30–60 min (estimate)

Nothing reached for the network. `age`, `tar`, `psql` and `node` are local, and the Postgres image was already pulled. On a machine that has never pulled it, that image (~1 GB) is the **one** thing §4 needs a connection for — so pull it while things are calm, not during the emergency.

8. Where things are

`MooTamer/coditu` (private; every clone is a whole copy) · `MooTamer/coditu-recovery` (self-contained recovery tooling, so recovery never starts by checking out the thing that was lost) · Vercel Hobby `coditu`, `https://coditu-phi.vercel.app` · Supabase Free `hayfsvsepgfescojesbd`, eu-central-1 · longer runbook `docs/12-RECOVERY-RUNBOOK.md` · what each provider is worth `docs/14-DEPENDENCY-LEDGER.md`.